

UNIVERSIDAD TÉCNICA NACIONAL

Sede San Carlos · ISW-521 Programación en Ambiente Web I

Prof. Bryan Miguel Chaves Salas

Reporte Técnico de Auditoría

*HTML5 Avanzado — Arquitectura Limpia Web &
Refactorización Semántica del Código*

Estudiante(s)	Wilberth Guillermo Guillen Aguilar
Fecha de entrega	2/6/2026
Modalidad	Individual
Archivo auditado	v1_base_feria_innovacion.html

1. Introducción

El presente reporte documenta el proceso de auditoría técnica realizado sobre el archivo [v1_base_feria_innovacion.html](#), correspondiente al sitio web oficial de la Feria Nacional de Innovación Estudiantil 2025. Este archivo fue desarrollado por un programador junior utilizando inteligencia artificial de manera acrítica, lo que resultó en un código funcional a simple vista pero técnicamente deficiente en su estructura y arquitectura.

Como desarrollador asumiendo el rol de Tech Lead en este escenario, mi responsabilidad fue revisar el código desde una perspectiva profesional, identificando todos aquellos problemas que comprometen la carencia de una arquitectura limpia. El análisis no se limitó a verificar si el código "funciona en el navegador", sino a evaluar si cumple con los estándares modernos de HTML5 y las buenas prácticas que cualquier equipo de desarrollo profesional esperaría encontrar en un entregable real.

El reporte está dividido en tres fases. La primera corresponde a la auditoría técnica del código original, donde se identifican y justifican seis categorías de errores con evidencia directa del código fuente. La segunda fase consiste en la construcción de un prompt técnico de alta precisión para guiar a una inteligencia artificial en la refactorización correcta del código. La tercera fase documenta la validación del resultado obtenido, tanto mediante el validador oficial del W3C como mediante inspección manual del DOM con las herramientas de desarrollo del navegador.

Este trabajo me permitió comprender que generar código con IA y saber revisarlo y comprenderlo son habilidades completamente distintas, y que la segunda es la que realmente define a un desarrollador con criterio profesional.

2. Resumen Ejecutivo — Errores Identificados

Se identificaron 6 categorías de errores arquitectónicos en el archivo. La siguiente tabla presenta la clasificación por severidad desde una perspectiva de arquitectura limpia:

#	Categoría de Error	Severidad	Descripción breve
1	Div-Soup — Ausencia semántica	CRÍTICO	Toda la estructura usa <div> genéricos. Viola el contrato semántico HTML5.
2	Formulario sin arquitectura	ALTO	Sin <label>, <fieldset>, ni <legend>. Formulario acoplado y no accesible.
3	Tabla sin estructura formal	ALTO	Falta <thead>, <tbody>, <tfoot>, <caption> y atributo scope en <th>.
4	Elemento <video> incompleto	MEDIO	Sin poster, controls ni fuente alternativa (.webm). Mala UX y CLS elevado.
5	<iframe> inseguro	ALTO	Sin loading, sandbox ni title. Vulnerabilidad de seguridad y rendimiento.
6	Anidamiento ilegal de elementos	CRÍTICO	Elementos de bloque dentro de elementos en línea. Viola el modelo de contenido HTML5.

3. Fase 1 — Auditoría Técnica con Enfoque de Arquitectura Limpia

3.1 Error #1 — Div-Soup: Violación del Contrato Semántico

¿Qué es el Div-Soup?

El "Div-Soup" se considera como antipatrón arquitectónico donde toda la estructura de un documento HTML se construye exclusivamente utilizando puros <div> genéricos, ignorando el sistema de etiquetas semánticas de HTML5. Desde la perspectiva de Arquitectura Limpia, esto equivale a tener todas las capas de una aplicación en un solo archivo monolítico sin separación de responsabilidades.

Impacto arquitectónico

- SEO degradado: Afectaría progresivamente el tráfico de usuarios, ya que la pérdida de posiciones en motores de búsqueda como Google reduciría drásticamente la visibilidad de la web, provocando la ubicación a páginas de resultados que rara vez son visitadas.
- Mantenibilidad crítica: Al carecer de una estructura clara, la legibilidad de un código no es la mas efectiva. Cuando el equipo de desarrollo crece o nuevos ingenieros se integran al proyecto, se vuelve imposible identificar la función o el propósito de un bloque de código a simple vista
- Acoplamiento estructura-estilo: Al depender exclusivamente de clases personalizadas (como .encabezado y .menu). Esto genera una fuerte dependencia entre si, donde este provoca un código con alto acoplamiento, si el sistema cambia ya sea estilos en CSS, este provocara problemas como reescribir los estilos tanto como la estructura del código html.
- Evidencia del código

```
<div class="encabezado">
  <div>
    <h1>Feria Nacional de Innovación Estudiantil 2025</h1>
    <div>ITCR Campus Cartago - 22, 23 y 24 de Octubre 2025</div>
    <div>Creando soluciones para los retos del mañana</div>
  </div>
</div>

<div class="menu">
  <a href="#inicio">Inicio</a>
  <a href="#proyectos">Proyectos</a>
  <a href="#agenda">Agenda</a>
  <a href="#inscripcion">Inscripción</a>
  <a href="#sede">Sede</a>
</div>

<div class="contenido">
  <div id="inicio" class="tarjeta">
    <h2>Acerca de la Feria</h2>
    <div>
      <p>
        La Feria Nacional de Innovación Estudiantil es el espacio anual donde estudiantes de todo el país presentan sus proyectos de investigación aplicada, prototipos tecnológicos y propuestas de emprendimiento ante un jurado de expertos nacionales e internacionales.
      </p>
      <p>
        Esta edición centra su atención en cuatro categorías: Robótica y Automatización, Biotecnología, Ciudades Inteligentes y Economía Circular.
      </p>
    </div>
  </div>
</div>
```

Etiquetas semánticas completamente ausentes

Etiqueta	Reemplaza a	Función arquitectónica
<header>	<div class="encabezado">	Encabezado de la página, puede tener: logo, título, slogan.
<nav>	<div class="menu">	Agrupar los enlaces de navegación principal del documento.

<main>	<code><div class="contenido"></code>	Contenido central, único por página.
<footer>	<code><div class="pie"></code>	Pie de la página donde incluye información institucional, derechos, contacto
<section>	<code><div class="tarjeta"></code>	Agrupar contenido que este dentro de un contexto relacionado.
<article>	Tarjetas de proyectos	Contenido autónomo y reutilizable (cada proyecto).

3.2 Error #2 — Formulario sin Arquitectura Semántica

Análisis arquitectónico

Un formulario HTML bien diseñado bajo principios de Arquitectura Limpia debe cumplir tres requisitos: lo que es la agrupación lógica de campos, donde haya una vinculación explícita entre etiqueta y campo para tener un bajo acoplamiento y uso del elemento más apropiado para cada tipo de dato.

El formulario del código original falla en los tres. No tiene `<label>` vinculados, no tiene `<fieldset>/<legend>` para agrupar, y usa `<select>` donde un `<datalist>` sería superior, porque recordemos que con un select es la lista fija de acuerdo al tema que este, pero con un datalist podemos tener varias sugerencias de diferentes tipos de temas.

Problema 1: Ausencia de `<label>` — Alto Acoplamiento

Cuando tenemos un campo de formulario, se necesita dos cosas: la etiqueta que describe el campo (lo que el usuario lee) y el campo en sí donde escribe. El problema es que en el código original esas dos cosas no están conectadas entre sí. Porque el `<div>` y el `<input>` son dos elementos totalmente independientes, el navegador en sí no sabe que uno describe al otro, lo correcto sería el `for="nombre-equipo"` del label y el `id="nombre-equipo"` del input crean un vínculo explícito, porque es importante porque cuando un `<label>` está correctamente vinculado a su campo, se puede hacer clic en el texto de la etiqueta y el cursor salta automáticamente al campo. Esto es especialmente útil.

Evidencia del código

```

<div>Nombre del equipo o proyecto:</div>
<input type="text" placeholder="Nombre oficial del proyecto">

<div>Nombre del responsable principal:</div>
<input type="text" placeholder="Nombre y apellidos completos">

<div>Correo de contacto:</div>
<input type="text" placeholder="correo@institucion.ac.cr">

<div>Número de teléfono:</div>
<input type="text" placeholder="XXXX-XXXX">

```

Problema 2: Sin <fieldset> ni <legend> — Ausencia de agrupación semántica

Arquitectónicamente, un formulario largo debe dividirse en grupos lógicos con responsabilidad única, similar al principio de alta cohesión en diseño de software. Los dos grupos que deberían existir son:

- Grupo 1 — Datos del equipo: nombre del proyecto, responsable, correo, teléfono, institución.
- Grupo 2 — Detalles del proyecto: carrera, categoría, descripción, requerimientos.

Evidencia del código

```
<div>Nombre del equipo o proyecto:</div>
<input type="text" placeholder="Nombre oficial del proyecto">

<div>Nombre del responsable principal:</div>
<input type="text" placeholder="Nombre y apellidos completos">

<div>Correo de contacto:</div>
<input type="text" placeholder="correo@institucion.ac.cr">

<div>Número de teléfono:</div>
<input type="text" placeholder="XXXX-XXXX">

<div>Institución de procedencia:</div>
<select>
```

Problema 3: <select> vs <datalist> — Principio de componente correcto

El <select> obliga al usuario a elegir solo entre las opciones predefinidas (se puede entender como una lista cerrada). El <datalist> crea una lista abierta: el usuario puede elegir una opción sugerida O escribir una carrera no listada. Para el campo de carrera académica, esta flexibilidad es arquitectónicamente superior porque refleja mejor la realidad (ejemplo: existen cientos de carreras posibles).

Evidencia del código

```
<div>Carrera o programa académico:</div>
<select>
  <option>Seleccione su carrera</option>
  <option>Ingeniería en Computación</option>
  <option>Ingeniería Eléctrica</option>
  <option>Ingeniería Ambiental</option>
  <option>Biotecnología</option>
  <option>Diseño Industrial</option>
  <option>Administración de Empresas</option>
  <option>Agronomía</option>
  <option>Arquitectura</option>
  <option>Otra</option>
</select>
```

3.3 Error #3 — Tabla de Agenda sin Estructura Formal

Análisis arquitectónico

Una tabla HTML bien diseñada no es solo una cuadrícula visual, es un modelo de datos con encabezados, cuerpo y resumen. Sin esta estructura, la tabla es datos sin contrato: el navegador no sabe qué es encabezado, qué es dato y qué es resumen.

Elementos estructurales faltantes y su función

Elemento	Tipo	Función y por qué no es solo estética
<caption>	Semántico	Describe el propósito de la tabla para lectores de pantalla y SEO.
<thead>	Estructural	Marca la fila de encabezados. El navegador puede repetirla en paginación.
<tbody>	Estructural	Conocido como el cuerpo de datos, se puede utilizar par agrupar filas.
<tfoot>	Estructural	Contiene resúmenes o totales. En la tabla actual: la fila de 'Total horas' debería estar aquí.

Evidencia del código

```
<table>
  <tr>
    <th>Hora</th>
    <th>Miércoles 22 Oct</th>
```

3.4 Error #4 — Elemento <video> Incompleto

Atributos faltantes y su impacto

Atributo faltante	Valor requerido	Impacto de su ausencia
poster	"img/portada-video.jpg"	Sin imagen previa, el elemento aparece como un bloque negro hasta que el usuario interactúa.
controls	(atributo booleano)	Sin controls, el usuario no puede pausar, ajustar volumen ni ver la duración. El video no puede reproducirse desde la interfaz. Problema crítico de usabilidad.
2° <source> (.webm)	type="video/webm"	Firefox y Chrome priorizan WebM por mejor compresión. Sin fuente alternativa, si el MP4 falla en algún navegador, el video simplemente no carga. Viola el principio de compatibilidad de tipos de formatos de video.

Evidencia del código

```
<video width="100%">
  <source src="video/feria_presentacion.mp4" type="video/mp4">
</video>
```

3.5 Error #5 — <iframe> Inseguro y Sin Optimización

Análisis de seguridad y rendimiento

Insertar un mapa o video usando un <iframe> es como abrir una ventana para mostrar otra página dentro de la tuya. Si lo dejas por defecto, esa página externa hereda los mismos permisos que tu web: podría ejecutar códigos raros o leer datos de tus usuarios, rompiendo la regla de "dar solo los permisos justos".

Para solucionarlo, aplicamos un Sandbox (Caja de Arena) directamente en el código usando el atributo sandbox.

El Sandbox actúa como un muro de cristal blindado: encierra la página externa en un entorno aislado para que se muestre en tu web, pero sin dejar que toque tus archivos, robe información.

Atributo faltante	Valor correcto	Justificación técnica
loading	"lazy"	Sin lazy loading, el mapa se descarga al cargar la página, aunque el usuario nunca llegue a esa sección. Penaliza el LCP (Largest Contentful Paint) del sitio.
sandbox	"allow-scripts allow-same-origin"	Sin sandbox, el iframe hereda todos los permisos de la página. Con sandbox se aplica Principio de Mínimo Privilegio: solo scripts y same-origin para que el mapa funcione, nada más.
title	"Mapa de ubicación del ITCR"	Sin title, los lectores de pantalla anuncian el iframe como "iframe" sin contexto. Requerido por WCAG 2.1 criterio 4.1.2.

Evidencia del código

```
<iframe
  src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3929.4567891234!2d-83.9134567!3d9.8634567!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x0%3A0x0!2zOckwNTEnNDgu
  width="100%"
  height="420"
  style="border:0;"
  allowfullscreen="">
</iframe>
```


3.6 Error #6 — Anidamiento Ilegal de Elementos (Violación del Modelo de Contenido)

Análisis arquitectónico

El navegador tiene reglas estrictas sobre qué etiquetas pueden ir dentro de otras. Romper estas reglas confunde al navegador; aunque intente arreglarlo solo, el diseño visual se puede romper de formas inesperadas.

Las etiquetas se dividen en dos grupos:

Elementos de bloque (Cajas grandes): Como `<div>`, `<section>`, `<p>` o los títulos (`<h1>`). Ocupan todo el ancho de la página y crean bloques completos de contenido.

Elementos en línea (Texto o detalles): Como `<span>`, `<a>` (enlaces) o `<strong>` (negritas). Se usan para partes específicas dentro de un texto.

La Regla: Una etiqueta en línea (pequeña) NUNCA puede contener a una etiqueta de bloque (grande).

Instancias identificadas en el código

Caso 1 — `<div>` y `<h3>` dentro de `<span>`

```
<span>  
  <div>  
    <h3>Secretaría de la Feria</h3>  
  </div>  
</span>
```

Caso 2 — `<div>` dentro de `<a>`

Nota técnica: en HTML5 permiten técnicamente meter un `<div>` dentro de un enlace `<a>` en algunos casos, en la práctica sigue siendo una mala idea que se evita por el impacto negativo que genera en el usuario. El principal problema es que al meter una caja grande dentro de un enlace se crea una zona fantasma para hacer clic, lo que significa que todo ese cuadro se vuelve interactivo y si un usuario está navegando desde el celular puede tocar por accidente cualquier espacio en blanco o vacío, siendo redirigido a otra página sin querer. Además, esto causa una gran confusión para las personas ciegas porque los lectores de pantalla que leen la web en voz alta no logran descifrar con exactitud dónde empieza y dónde termina realmente el enlace, haciendo que la navegación sea muy frustrante.

```
<a href="https://www.tec.ac.cr">  
  <div>  
    <div>Portal oficial del ITCR</div>  
  </div>  
</a>
```

Cómo detectarlo con DevTools

- Abrir el archivo en Chrome o Edge y presionar F12.

- En la pestaña Elements, buscar el árbol DOM(estructura interna del navegador) generado por el navegador.
- Si el navegador "corrigió" el anidamiento ilegal, verá que los elementos fueron reorganizados automáticamente (el <div> aparece fuera del , por ejemplo).
- La diferencia entre el HTML original y el DOM resultante es que revela los errores de anidamiento: donde el navegador movió elementos es donde el código violaba el modelo de contenido.

4. Fase 2 — Ingeniería de Prompts para Refactorización

A continuación se presenta la estructura del prompt técnico de alta precisión. Complete cada sección con el texto que usted construyó.

Sección A — Rol y Contexto

"Actúa como un Desarrollador Frontend Senior con 10 años de experiencia en desarrollo web profesional, certificado en estándares W3C y especializado en HTML5 semántico, arquitectura limpia web y accesibilidad (WCAG 2.1). Tu criterio técnico está por encima de cualquier convención visual: para vos, un documento HTML mal estructurado semánticamente es tan inaceptable como un sistema sin separación de capas. Conocés a fondo el modelo de contenido HTML5, las métricas Core Web Vitals, y las mejores prácticas de seguridad en contenido embebido."

Sección B — Descripción del Problema

"A continuación te entrego el código fuente del archivo v1_base_feria_innovacion.html, correspondiente al sitio web oficial de la Feria Nacional de Innovación Estudiantil 2025. Este archivo fue generado por un programador junior utilizando inteligencia artificial sin criterio técnico, por lo que aunque el sitio se visualiza correctamente en el navegador, su estructura interna presenta graves deficiencias arquitectónicas."

Los problemas identificados son los siguientes:

- 1. Div-Soup total: toda la estructura del documento está construida con etiquetas <div> genéricas. No existe ninguna etiqueta semántica HTML5 como <header>, <nav>, <main>, <footer>, <section> ni <article>.*
- 2. Formulario sin arquitectura semántica: los campos no tienen <label> vinculados mediante for e id, no existe agrupación mediante <fieldset> y <legend>, y el campo de carrera usa un <select> donde debería usarse un <datalist>.*
- 3. Tabla de agenda incompleta: la tabla carece de <caption>, <thead>, <tbody> y <tfoot>, y ningún <th> tiene el atributo scope definido.*
- 4. Elemento <video> deficiente: no tiene atributo poster, no tiene controls, y solo cuenta con una fuente en formato MP4 sin alternativa en WebM.*
- 5. iframe inseguro y sin optimización: el mapa embebido no tiene loading='lazy', carece del atributo sandbox y no tiene title descriptivo.*

6. Anidamiento ilegal de elementos: en la sección de contacto hay elementos de bloque como `<div>` y `<h3>` anidados dentro de elementos de línea como `<span>` y `<a>`, violando el modelo de contenido HTML5."

Sección C — Restricciones Técnicas (mínimo 8)

"Las siguientes restricciones son obligatorias. Debes cumplirlas todas sin excepción:

1. DEBES reemplazar `<div class="encabezado">` por `<header>`, `<div class="menu">` por `<nav>`, `<div class="contenido">` por `<main>` y `<div class="pie">` por `<footer>`. Las tarjetas de contenido temático deben usar `<section>` y cada proyecto finalista debe usar `<article>`.
2. DEBES dividir el formulario en exactamente dos grupos usando `<fieldset>` y `<legend>`. El primer grupo llamado "Datos del Equipo" debe contener: nombre del proyecto, responsable, correo y teléfono. El segundo grupo llamado "Detalles del Proyecto" debe contener: institución, carrera, categoría, descripción y requerimientos del stand.
3. DEBES añadir `<label for="id">` a cada campo del formulario, donde el valor de `for` debe coincidir exactamente con el `id` del `input` correspondiente. Ningún campo puede quedar sin su label vinculado.
4. DEBES eliminar el `<select>` del campo de carrera y reemplazarlo por un `<input type="text" list="lista-carreras">` vinculado a un `<datalist id="lista-carreras">` que contenga las mismas opciones que tenía el `select` original.
5. DEBES modificar el elemento `<video>` añadiendo el atributo `poster` con valor `"img/portada-video.jpg"`, el atributo `controls`, y una segunda fuente `<source src="video/feria_presentacion.webm" type="video/webm">` antes del cierre de la etiqueta.
6. DEBES modificar el `<iframe>` del mapa añadiendo `loading="lazy"`, `sandbox="allow-scripts allow-same-origin"` y `title="Mapa de ubicación del ITCR Campus Cartago"`.
7. DEBES corregir el anidamiento ilegal en la sección de contacto: el `<span>` que contiene `<div>` y `<h3>` debe reemplazarse por `<address>`. El enlace `<a>` que contiene `<div>` debe reestructurarse usando únicamente elementos de línea en su interior.
8. DEBES añadir `<caption>Agenda de la Feria Nacional de Innovación Estudiantil 2025</caption>` como primer hijo de la tabla, envolver la fila de encabezados en `<thead>`, los datos en `<tbody>` y la fila de totales en `<tfoot>`. Todos los `<th>` deben tener `scope="col"`.
9. DEBES añadir el atributo `lang="es"` al elemento `<html>` y verificar que el botón de envío tenga `type="submit"` explícito."
- "10. DEBES respetar la jerarquía de encabezados en orden de cascada. El documento debe tener un único `<h1>` como título principal, los títulos de cada sección deben usar `<h2>`, y los subtítulos dentro de cada sección deben usar `<h3>`. Está prohibido saltar niveles, por ejemplo pasar de `<h1>` directamente a `<h3>` sin un `<h2>` intermedio, o usar `<h3>` antes de que exista un `<h2>` en esa sección. La jerarquía de encabezados debe reflejar la estructura lógica del documento, no el tamaño visual del texto."

Sección D — Criterios de Validación

"Antes de entregarme el resultado, debes verificar tu propio código revisando cada punto de esta lista. Si alguno no se cumple, corrígelo antes de responder:

- 1. ¿El `<body>` tiene como hijos directos `<header>`, `<nav>`, `<main>` y `<footer>`? ¿No queda ningún `<div>` cumpliendo función estructural?*
- 2. ¿El formulario tiene exactamente dos `<fieldset>`, cada uno con su `<legend>` descriptivo? ¿Todos los campos tienen `<label for="...">` vinculado al id de su input correspondiente?*
- 3. ¿El campo de carrera usa `<input type="text" list="...">` conectado a un `<datalist>`? ¿El `<select>` original fue eliminado completamente?*
- 4. ¿El `<video>` tiene los atributos `poster` y `controls`, y contiene al menos dos `<source>`: uno MP4 y uno WebM?*
- 5. ¿El `<iframe>` tiene `loading="lazy"`, `sandbox="allow-scripts allow-same-origin"` y un `title` descriptivo?*
- 6. ¿La sección de contacto no contiene ningún elemento de bloque dentro de un elemento de línea? ¿El `<span>` fue reemplazado por `<address>`?*
- 7. ¿La tabla tiene `<caption>`, `<thead>`, `<tbody>` y `<tfoot>`? ¿Todos los `<th>` tienen el atributo `scope` definido?*
- 8. ¿El documento tiene un único `<h1>` como título principal? ¿Los títulos de sección usan `<h2>` y los subtítulos usan `<h3>`? ¿No existe ningún salto de nivel, por ejemplo un `<h3>` sin un `<h2>` previo en esa misma sección?*
- 9. ¿El elemento `<html>` tiene `lang="es"` y el botón de envío tiene `type="submit"`?*
- 10. ¿El código resultante es HTML5 válido que pasaría el validador de W3C sin errores críticos?"*

Sección E — Formato del Resultado

"Entregame únicamente el código HTML completo y funcional, listo para guardarse como archivo `v2_feria_refactorizado_final.html`. El resultado debe cumplir las siguientes condiciones de formato:

- 1. El código debe iniciar con `<!DOCTYPE html>` y cerrar con `</html>`. Sin texto introductorio antes ni después.*
- 2. No incluyas explicaciones, comentarios hacia mí, ni resúmenes de los cambios realizados. Solo el código.*
- 3. No uses bloques de código Markdown como ````html`. Entrega el HTML directamente.*
- 4. El código debe estar correctamente indentado con 4 espacios por nivel para garantizar legibilidad.*

5. Si consideras necesario incluir comentarios dentro del código HTML para identificar secciones, estos deben seguir el formato `<!-- Nombre de la sección -->` y ubicarse únicamente al inicio de cada sección principal."

5. Fase 3 — Revisión Final del DOM

5.1 Validación con el Validador W3C

URL: https://validator.w3.org/#validate_by_input

1. **Error** Bad value '100%' for attribute 'width' on element 'video': Expected a digit but saw '%'. Instead.
From line 251, column 13 to line 255, column 103
`<video>` `controls` `poster="img/portada-video.jpg"` `width="100%"` `aria-label="Video de presentación de la Feria Nacional de Innovación Estudiantil 2025"`

2. **Error** Bad value '100%' for attribute 'width' on element 'iframe': Expected a digit but saw '%'. Instead.
From line 497, column 13 to line 505, column 32
`<iframe>` `title="Mapa de ubicación del ITCR Campus Cartago"` `src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3929.4567891234!2d-83.913.768!4f13!1s0x83A0x0!2zOCKwITENMDgucj0IDgzwA1NCc0C0401c!5e0!3m2!1ses!2scr!4v9876543210"` `width="100%"` `height="420"` `loading="lazy"` `sandbox="allow-scripts allow-same-origin"` `referrerpolicy="no-referrer-when-downgrade"` `allowfullscreen`

3. **Warning** Potentially bad value 'allow-scripts allow-same-origin' for attribute 'sandbox' on element 'iframe': Setting both 'allow-scripts' and 'allow-same-origin' is not recommended, because it effectively enables an embedded page to break out of all sandboxing.
From line 497, column 13 to line 505, column 32
`<iframe>` `title="Mapa de ubicación del ITCR Campus Cartago"` `src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3929.4567891234!2d-83.913.768!4f13!1s0x83A0x0!2zOCKwITENMDgucj0IDgzwA1NCc0C0401c!5e0!3m2!1ses!2scr!4v9876543210"` `width="100%"` `height="420"` `loading="lazy"` `sandbox="allow-scripts allow-same-origin"` `referrerpolicy="no-referrer-when-downgrade"` `allowfullscreen`

4. **Error** The element 'h3' must not appear as a descendant of the 'address' element.
From line 513, column 17 to line 513, column 20
`<h3>Secret`

Error 1 — width="100%" en <video>

El atributo width en un elemento <video> solo acepta valores numéricos enteros como width="800". El porcentaje 100% no es un valor válido para ese atributo HTML, ese tipo de dimensionamiento debe hacerse desde CSS, no desde el atributo HTML.

Corrección: eliminar width="100%" del elemento <video> y controlarlo desde el CSS que ya existe en el archivo con la regla video { width: 100%; }.

Error 2 — width="100%" en <iframe>

Mismo problema que el anterior. El atributo width del <iframe> tampoco acepta porcentajes, solo números enteros.

Corrección: eliminar width="100%" del <iframe> y manejarlo desde CSS con iframe { width: 100%; }.

Warning 3 — sandbox="allow-scripts allow-same-origin" en <iframe>

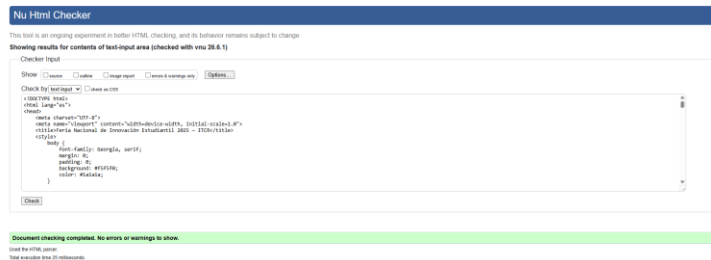
El validador advierte que usar allow-scripts y allow-same-origin juntos en el sandbox es potencialmente peligroso, porque combinados permiten que el contenido embebido escape el sandbox ejecutando scripts con los permisos del origen padre.

Corrección: para Google Maps basta con usar solo sandbox="allow-scripts" que es suficiente para que el mapa funcione sin el riesgo de escape.

Error 4 — <h3> dentro de <address>

El elemento <address> en HTML5 no puede contener encabezados como <h3>. Su modelo de contenido solo permite contenido de fraseo, no elementos de flujo con jerarquía semántica.

Corrección: sacar el <h3>Secretaría de la Feria</h3> fuera del <address> y ponerlo como hermano anterior.



#con corrección

5.2 Inspección del DOM con DevTools

Preguntas de verificación — responda cada una

- ¿El <header>, <nav>, <main> y <footer> están presentes como hijos directos del <body>?

Sí. Al revisar el árbol del DOM en la pestaña Elements confirmé que los cuatro elementos semánticos principales aparecen como hijos directos del <body> en el orden correcto: primero el <header>, luego el <nav aria-label="Navegación principal">, seguido del <main> y finalmente el <footer>. Esto confirma que la refactorización logró reemplazar correctamente los <div> estructurales originales por etiquetas con responsabilidad semántica única, tal como lo establece el estándar HTML5.

¿Los <section> y <article> contienen únicamente elementos de bloque?

Sí. Dentro del <main> pude verificar que todos los <section> y <article> presentes contienen únicamente elementos de bloque en su interior. No encontré ningún caso donde un elemento de fraseo como o <a> estuviera actuando como contenedor de bloques dentro de ellos, lo que confirma que el modelo de contenido HTML5 se respeta en su totalidad en esta versión.

¿No hay ningún <div> o <p> que sea hijo directo de un o <a>?

Correcto. No encontré ningún caso de anidamiento ilegal en el árbol DOM. En la versión original, el error estaba en la sección de contacto donde un contenía directamente un <div> y un <h3>, violando el modelo de contenido HTML5. En la versión refactorizada ese bloque fue corregido utilizando <address> como contenedor semántico apropiado, eliminando por completo la violación.

¿ Contraste entre la versión original y la refactorizada:?

La diferencia en la jerarquía del DOM entre ambas versiones es muy notable. En la versión original, el árbol del <body> estaba formado únicamente por <div> genéricos apilados sin ningún significado estructural, lo que hacía imposible que el navegador, los motores de búsqueda o los lectores de pantalla pudieran identificar las zonas del documento sin leer el CSS asociado. En la versión refactorizada, en cambio, el árbol del <body> presenta una jerarquía limpia y autoexplicativa: el <header> como zona de identidad del sitio, el <nav> como sistema de navegación principal, el <main> como contenedor del contenido relevante con sus <section> y <article> internos bien organizados, y el <footer> como cierre institucional. Esta transformación convierte un documento plano y ambiguo en uno con arquitectura semántica clara, donde cada elemento comunica su propósito por sí solo.

5.3 Checklist de Verificación Final

✓	Criterio de verificación
Y	La página usa <header>, <nav>, <main>, <footer> en lugar de <div> estructural
Y	El contenido principal se divide en <section> o <article> con semántica apropiada
Y	El formulario tiene al menos dos <fieldset> con <legend> descriptivos
Y	Todos los <label> usan for vinculado al id de su campo correspondiente
Y	El campo de carrera usa <input list='...'> + <datalist> (NO un <select>)
Y	El <video> tiene poster, controls y mínimo dos <source> con distintos type
Y	El <iframe> tiene loading='lazy', sandbox con valores apropiados y title
Y	NO hay elementos de bloque (<div>, <p>, <h1>-<h6>) dentro de o <a>
Y	La tabla tiene <caption>, <thead>, <tbody>, <tfoot> y scope en todos los <th>
Y	La validación W3C no reporta errores (solo advertencias menores son aceptables)
Y	El elemento <html> tiene atributo lang='es'
Y	El botón submit tiene atributo type='submit'

6. Reflexión Final — IA como Herramienta, no como Arquitecto

Antes de este laboratorio, mi percepción sobre usar inteligencia artificial para generar código era bastante positiva. Lo veía como una herramienta que acelera el trabajo y reduce el esfuerzo. Sin embargo, al ponerme en el rol de Tech Lead y auditar el código que una IA produjo, esa percepción cambió completamente. Lo que más me sorprendió fue darme cuenta de que el código generado funcionaba visualmente en el navegador, pero por dentro era una estructura frágil y mal construida. Todo se veía bien a simple vista, pero al revisarlo línea por línea encontré que no había ni una sola etiqueta semántica correcta, el formulario no tenía ningún vínculo entre sus etiquetas y sus campos, la tabla era básicamente una cuadrícula sin estructura formal, y el iframe estaba expuesto sin ninguna restricción de seguridad. La IA nunca se detuvo a preguntarse si lo que generaba era correcto arquitectónicamente, simplemente produjo algo que "parecía funcionar".

Esa diferencia es enorme. Codificar con IA es rápido, pero requiere que quien revisa el resultado tenga criterio técnico real para detectar lo que la herramienta no sabe que está haciendo mal. Sin esa revisión crítica, el código generado por IA puede convertirse en deuda técnica que crece silenciosamente hasta que un proyecto entero se vuelve difícil de mantener. Este laboratorio me dejó claro que la IA es útil como punto de partida, pero la responsabilidad del resultado siempre recae en el desarrollador que firma el entregable.